

Analýza a porovnanie game engineov CryEngine, Unity a Unreal engine

Michal Boroš

Slovenská technická univerzita v Bratislave Fakulta
informatiky a informačných technológií Ústav informatiky,
informačných systémov a softvérového inžinierstva Ilkovičova

2, 842 16 Bratislava 4

December 17, 2023

Abstrakt

Hranie počítačových hier alebo gaming je každým dňom viac dostupný a tak isto populárny. Ľudia všetkých vekových kategórii si dokážu oddýchnuť alebo sa zabaviť pri hraní počítačových hier. Vytvorenie počítačovej hry je avšak celkom zložitý proces, keďže vývojári musia vytvárať (napísať) všetko od nuly. Tu však prichádzajú na svet game engines v preklade herné stroje, ktoré vývojárom značne zjednodušia tento proces. Existuje mnoho game engineov napríklad Unity, CryEngine, Build Box, Godot, Unreal Engine a iné. Táto práca sa bude venovať analýze architektúry game engineov.

1 Úvod

Pre vývoj počítačových hier je game engine priam až potrebou. Preto je veľmi dôležité vybrať si ten správny game engine pre svoju potrebu. V tomto článku sa najprv pozrieme na analýzu game engine ako takého, kde sa pozrieme a analyzujeme jeho časti. Tieto časti sú napríklad fyzický modul, vykresľovací modul, audio modul, scripting modul, a iné.

Ďalej sa pozrieme na silné stránky jednotlivých game engineov a spravíme porovnanie medzi nimi. Na záver sme sa pokúsili vytvoriť napodobeninu jednoduchej dvoj rozmernej hry flappy bird. Túto hru sme vytvorili bez game engineu v programovacom jazyku Python a knižnicou pygame, a vytvorili sme ju rovnako aj pomocou game engineu unity, ktorý je hlásaný ako jeden z najlepších na jednoduché, mobilné hry. Budeme porovnávať jednotlivé procesy vytvorenia hry, od vytvárania objektov, prostredia, detekovanie kolízií, aplikovanie gravitácie, pridávanie zvukov, jednoduchosť používania obrázkov, a tak ďalej.

2 Architektúra game engineov

2.1 Game engine

Game engine alebo v preklade herný stroj je počítačový framework vytvorený na zjednodušenie vývoja počítačových hier. Dáva vývojárom veľa užitočných nástrojov a knižníc na zjednodušenie tohto procesu, vytvárania video hier. Game enginey majú na starosti viaceré fundamentálne funkcionality pre normálny chod hry. Stará sa o vykresľovanie grafiky, aplikovanie fyziky, púšťanie zvuku, a mnoho ďalších ktoré budú spomenuté v sekcii neskôr. Často sú game enginey vo forme prostredia alebo editora v ktorom sa dajú herné elementy vy-

berať potiahnutím myšky niekedy úplne bez toho aby musel vývojár napísať bodku kódu.

Game enginey môžu byť v rôznych formách, niektoré vytvorené veľkými spoločnosťami, pri ktorých musí mať vývojár drahú licenciu. Naopak sú aj open source game enginey, ktoré dovoľujú vytvárať hry zadarmo a začínú si brať podiel zo zisku, až keď hra nadobudne nejaký úspech. Známe game enginey sú napríklad Unity, Unreal engine, CryEngine, Godot, Build Box, a mnoho iných.

Game engine dovoľuje vývojárovi pozeráť sa na hru z vyššieho nadhľadu a nemusí písať každý riadok kódu na nízkej úrovni. Medzi výhody patrí, že sa môže venovať vytváraniu príbehov, hernému dizajnu, alebo herným mechanikám.

Ďalšou výhodou game enginey je multi-platformovosť, kedy sa vývojár nemusí starať o to aby bola hra dostupná na iné platformy.

Teraz sa poďme pozrieť na nejaké časti game enginey.

2.1.1 Rendering engine

Rendering engine alebo modul na vykresľovanie je jednou zo základných súčastí game enginey. Stará sa v podstate o to aby boli všetky pixely na obrazovke tak ako majú byť. Používajú sa na to vykresľovacie knižnice ako OpenGL, DirectX alebo SDL. Je práca vykresľovacieho modulu, aby bola efektívna komunikácia medzi game engineom a grafickým hardvérom koncového používateľa. [6]

2.1.2 Physics engine

Physics engine alebo fyzikálny modul sa stará o všetku interakciu medzi hernými objektami. Detekcia kolízií medzi objektami je jednou z týchto interak-

cii. Ďalšou je napríklad aplikovanie síl na herné objekty, napríklad gravitácia.

2.1.3 Scripting engine

Väčšinou v game engine existujú základné veci, z ktorými si niektorí vývojári dokážu vystačiť. Sú to veci ako aplikovanie gravitácie na objekty, detekovanie kolízií, vykresľovanie hráča a tak ďalej. Pre pokročilejších vývojárov, tieto funkcie však nemusia stačiť. V tomto prípade si musí funkcie alebo skripty napísať sám. Každý game engine, podporuje iný programovací jazyk viď. tabuľka 1.

2.1.4 Umelá inteligencia

Umelá inteligencia sa využíva napríklad na správanie nehrateľných charaktarov NPC. Umelá inteligencia je nevyhnutná na vytvorenie pohlcujúceho prostredia pre hráča. Využívajú sa algoritmi na rozhodovanie, algoritmi na hľadanie cesty, stavové automaty.

2.1.5 Asset manažér

Asset v preklade majetok je braný v game engineoch ako všetko, čo môžeme v hre použiť. Od obrázkov, zvukov až po animácie. Ich cieľ je definovať ako bude hra vyzeráť a znieť keď beží. Väčšina game engineov má vlastné asset obchody, v ktorých vývojári a dizajnéri môžu predávať svoje digitálne výtvary.

2.2 CryEngine

CryEngine je game engine vytvorený spoločnosťou Crytek. Začali ho vyvíjať v roku 2002, a v roku 2004 bol presentovaný. Prvá hra vytvorená na CryEngine je FarCry. Od roku 2016, kedy vyšiel CryEngine V sa dá vytvá-

rať hocikaké hry nie len FPS (first person shooter). Má veľmi dobré grafické efekty, ale veľmi zlú dokumentáciu. [2, 8]

2.3 Unity

Unity bol vytvorený spoločnosťou Unity Technologies a vyšiel v roku 2005. Je populárny hlavne na mobilné a indie hry. Populárne hry vytvorené v Unity sú Pokemon Go alebo CoD: Mobile. Má trochu horšie grafické efekty ale za to má najlepšiu dokumentáciu. [1, 8]

2.4 Unreal Engine

Unreal engine je vyvíjaný spoločnosťou Epic Games. Prvá hra napísaná v ňom je Unreal (FPS), a vyšla v roku 1998. Má veľmi dobré grafické efekty, a celkom dobrú dokumentáciu. [3, 8]

2.5 Porovnanie

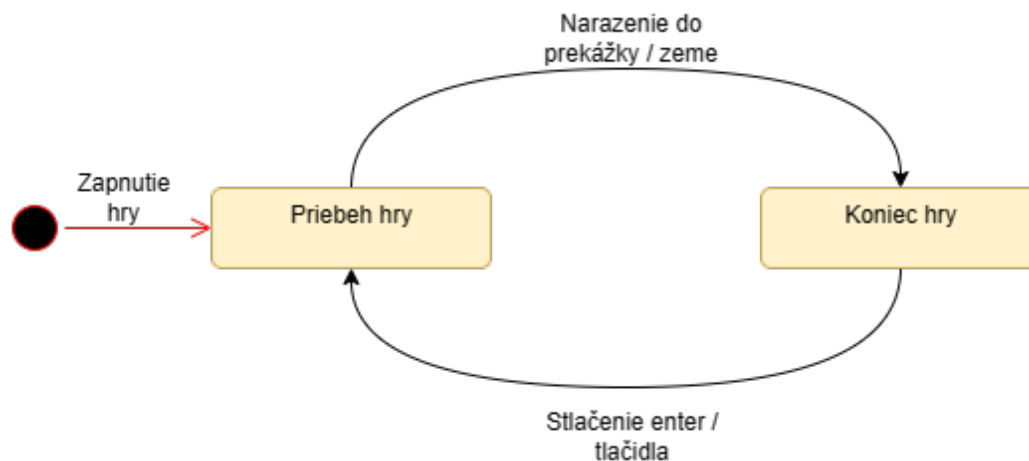
Parameters	Unity	Unreal	CryEngine
Cross Platform	Consoles, PC, mobile	Consoles, PC, mobile	Consoles, PC, mobile
OS Support	Windows, Linux, Mac	Windows, Mac	Windows, Linux
Programming languages	C#, JavaScript, Boo	C++	C++, Lua
Multifunctionality	2D, 3D	2D, 3D	3D
Documentation	Best	Good	Poor
Difficulty level	Low	High	High
Artificial Inteligence	RAIN	Kynapse	Lua driven AI
Physics Engine	PhysX	PhysX	Soft-body
Animations	Basic	Skeletal control and dynamic animation	Skeletal Control and Facial Editor

Tabuľka 1: Porovnanie aspektov game enginov Unity, Unreal Engine, a Cry-Engine [4]

3 Prípadová štúdia: komparácia prístupov

Vytvorili sme dva krát tú istú hru, v dvoch rôznych prostrediach. Hra ktorú sme nakódili je napodobnenina Flappy bird, Little fire in icy kingdom. Hra funguje rovnako ako flappy bird, takže hráč je plamienok, ktorý padá na zem kvôli gravitačnej sile. Hráč môže lietať alebo skôr skákať stlačením tlačidla medzerník. Týmto vertikálnym pohybom sa hráč musí vyhýbať prekážkam, ktoré sa hýbu z pravej strany obrazovky na ľavú. Hráč dostane 10 bodov za každú prekážku, ktorej sa úspešne vyhne a ak narazí hra skončí. Pri získaní

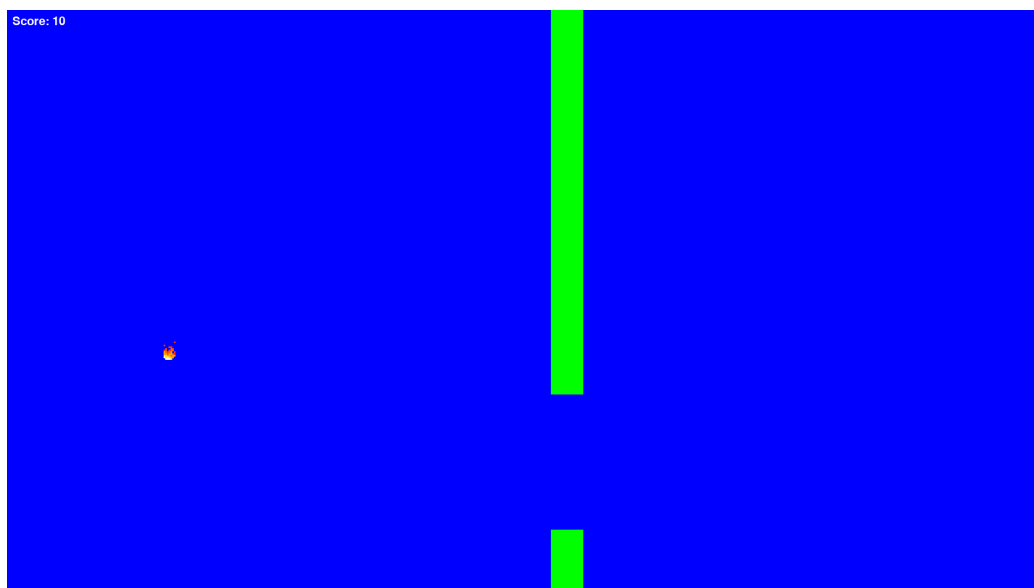
bodov hráč počuje krátku zvučku. Keď hra skončí môže hráč hru pustiť od znovu. Hry nie sú úplne totožné, ale z väčšiny sú identické.



Obr. 1: Stavový diagramy hry

3.1 Python pygame

Prvú implementáciu sme robili bez game engine, v programovacom jazyku Python. V Pythone existuje knižnica pygame, ktorá nám umožňuje robiť 2D hry jednoduchšie.



Obr. 2: Hra napísaná v Pythone - herná obrazovka



Obr. 3: Hra napísaná v Pythone - game over obrazovka

3.2 Unity game engine

Druhú implementáciu sme robili pomocou game engineu Unity. Unity nám poskytuje veľmi veľa silných nástrojov, na vytváranie 2D hier. Pre začiatočníkov, to je až moc veľa nástrojov, čo ale nie je až taký problém, lebo existuje veľmi veľa tutoriálov a dokumentácia je taktiež nad očakávania. Keďže som v Unity a všeobecne nejakom game engine robil prvý krát, postupoval som podľa tutoriálu od Valem[7].



Obr. 4: Hra napísaná v unity - herná obrazovka



Obr. 5: Hra napísaná v unity - game over obrazovka

3.3 Porovnanie

3.3.1 Hráč - plamienok

V pythone načítame obrázok, a z neho vytvoríme rect objekt, s ktorým sa potom manipuluje.

```
player_image =pygame.image.load("frame_0_delay-0.17s.png").  
                convert_alpha()  
player_image =pygame.transform.scale(player_image, (player_width,  
                player_height))  
player_rect =player_image.get_rect()  
player_rect.x =screen.get_width() /7  
player_rect.y =screen.get_height() /2 -500
```

V unity len potiahneme obrázok z assetov na scénu, a umiestnime ho tam kde chceme aby sa nachádzal.

3.3.2 Gravitácia a lietanie

V pythone všetko ovládame cez menenie y súradnice objektu, keď k y pričítame tak hráč klesá, a naopak keď od y odčítame tak hráč stúpa, čiže lieta. Hráč lieta po stlačení tlačidla space. V unity sa o gravitáciu stará komponent rigidbody 2D, ktorý sme pridali k nášmu hráčovi. Na lietanie sme museli vytvoriť skript, Fly.cs. Hráč lieta po stlačení tlačidla space.

3.3.3 Vytváranie prekážok

V pythone sa prekážky vytvárajú cez objekt rect. Po vytvorení prekážky sa prekážka pohybuje smerom do ľava, odčítavaním jej x-ovej súradnice. Keď prejde úplne na ľavú stranu, prepíše sa x-ová súradnica na začiatočnú, a zmení sa veľkosť hornej prekážky, aby neboli prekážky na rovnakom mieste. Medzera medzi prekážkami ostáva vždy rovnaká.

```
if obstacle_x <0 -60:  
    obstacle_x =screen.get_width()  
    obstacle_size =random.randint(25, screen.get_height() -gap)
```

V unity zase potiahneme prekážky na scénu, na to miesto kde chceme aby sa vytvárali. Potom spravíme nový herný objekt pipe spawner, ktorý vytvorí novú prekážku každé 4 sekundy. Po 15 sekundách sa prekážka zničí. Je tu iný prístup ako v pythone preto, lebo som sa riadil podľa tutoriálu a prišlo mi to ako zaujímavý prístup.

3.3.4 Kolízia

V pythone sme si vytvorili funkciu collision, ktorá sa pozerá, či sa náhodou nezrazil hráč s prekážkami, alebo zemou. Vracia sa stav hry 2 game over, 1 normálna hra.

```
def collision(player_rect, obstacle_rect):  
    if pygame.Rect.colliderect(player_rect, obstacle_rect[0]):  
        return 2  
    if pygame.Rect.colliderect(player_rect, obstacle_rect[1]):  
        return 2  
    if player_rect.y > screen.get_height() - player_height:  
        player_rect.y = screen.get_height() - player_height  
        return 2  
    return 1
```

V unity sa to rieši jednoduchšie, a to tak, že každému objektu pridáme komponent collider 2d. Potom k hráčovi do Fly.cs pridáme metódu OnCollisionEnter2D, ktorá sa volá pri kolízii automaticky.

```
private void OnCollisionEnter2D(Collision2D collision)  
{  
    gameManager.GameOver();  
}
```

3.3.5 Koniec Hry

V pythone máme premennú state, ktorá nadobúda hodnoty 1 a 2. Jeden znamená, že hra prebieha a dva znamená, že hra skončila. Po skončení vypíšeme na obrazovku Game Over. Po stlačení tlačidla enter sa znovu spustí hra. Preto sa musia všetky premenné vrátiť do pôvodného stavu. V unity sa pri kolízii zavolá funkcia GameOver a tým vypíše koniec hry, a zastaví hru. Na obrazovke je tlačidlo na reštartovanie hry.

3.3.6 Skóre

V pythone sa pozeráme, kedy hráč prejde cez prekážku, respektíve kedy prekážka prejde cez hráča, keďže hráč sa pohybuje len vertikálne. V unity si vytvoríme nový herný objekt v medzere medzi prekážkami, pridáme mu komponent collider 2d, ale nastavíme mu vlastnosť on trigger. Toto zariadi, že pri aktivovaní sa spustí skript AddScore.

3.3.7 Zvukové efekty

Zvukové efekty sa pridávajú veľmi podobne aj v pythone aj v unity. Len je syntax trochu iná, kvôli rozdielnym programovacím jazykom. Načíta sa zvuk ktorý sa má pustiť, a potom v momente kedy sa má prehrať sa prehra. V python je to `sound.play()` a v C# je to `sound.Play()`.

3.3.8 Exportovanie hry

Pre exportovanie hry do spustiteľného súboru musíme v pythone použiť nejakú knižnicu. Vybrali sme si knižnicu pyinstaller. Unity nám dokáže vygenerovať spustiteľný súbor stlačením jedného tlačidla.

4 Súvisiaca práca

Kanál Emerald sa vo videu I Made the Same Game in 8 Engines [5] venoval podobnému problému, ako táto práca. Vytvoril jednoduchú hru v 8 rôznych game enginech. Konkrétne to boli Unity, Godot, Unreal engine, GameMaker, Construct, GDevelop, RPG Maker a Scratch. Princíp hry je, že hráč sa vie hýbať zo strany na stranu a musí zbierať objekty padajúce z neba. Emerald pekne vysvetlil, výhody a nevýhody vytvárania tejto hry v rôznych game enginech.

V papieri od Barczaka a Wozniaka [4] sa venovali podobnému problému, porovnávaníu game enginov. Porovnávali ich na základe rôznych kritérií, ako animačný modul, fyzikálny modul, audio modul, umelá inteligencia, cena, dokumentácia, a dokonca aj výkonnosť. Vytvorili herný svet o veľkosti 1048 kilometrov štvorcových, a zaplnili ho rôznymi elementami z prírody, stromy, kríky a podobne. Potom zisťovali záťaž hry na rovnakom počítači.

5 Záver

V tejto práci sme sa venovali hlavne porovnaniu game enginov, a porovnaniu hry vytvorenej pomocou Unity, a pygame. V prvej časti sme analyzovali architektúru game enginu. Game engine sa skladá z istých komponentov ako sú napríklad fyzikálny modul, audio modul, asset manažér a iné.

V druhej časti sme porovnali game enginy Unity, CryEngine a Unreal Engine. Zistili sme, že každý engine má nejaké svoje výhody ale aj svoje nevýhody. Unity je výborný pre začiatočníkov, hlavne kvôli jeho vynikajúcej dokumentácii, a jednoduchosti. Avšak Unity nemá takú dobrú grafiku. CryEngine má výbornú grafiku, je vhodný na 3D hry hlavne FPS, ale má veľmi zlú dokumentáciu, takže začiatočníci by sa s ním trápili. Unreal Engine je veľmi zložitý na naučenie, ale za to sa v ňom vytvára väčšina AAA hier, kvôli jeho kvalite a funkcionalitám.

V tretej časti sme sa venovali implementovaniu jednoduchkej hry flappy birda v dvoch prostrediach, Unity a Python pygame. V oboch prístupoch sa nám hra podarila spraviť, a hry sú skoro identické, až na pár drobných rozdielov. Analyzovali sme jednotlivé komponenty hry, v oboch prostrediach. Časovo sa to nedá moc porovnať, keďže v Pythone mám skúsenosti, a v game engine som robil prvý krát. Ale to nebol problém, keďže existuje veľa

tutoriálov na Unity.

Literatúra

- [1] <https://unity.com/>.
- [2] <https://www.cryengine.com/>.
- [3] <https://www.unrealengine.com/en-us>.
- [4] A. Barczak and H. Woźniak. Comparative study on game engines. *Studia Informatica. System and information technology*, 23(1-2):5–24, Dec. 2020.
- [5] Emerald. I made the same game in 8 engines, <https://www.youtube.com/watch?v=masaqbrmnam>.
- [6] A. Thorn. *Game Engine Design and Implementation*. Jones and Bartlett Learning, 2010.
- [7] Valem. Make your own flappy bird in 10 minutes (unity tutorial), <https://www.youtube.com/watch?v=urwmejxy334>.
- [8] C. Vohera, H. Chheda, D. Chouhan, A. Desai, and V. Jain. Game engine architecture and comparative study of different game engines. In *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–6, 2021.